

Model Comparison: YOLOv8s (production) vs YOLO11x vs RT-DETR-x vs Co-DETR

Date: 2026-07-02 / 2026-07-03 **Test set:** `ml/data/combined/test` — 2,834 images, held-out split of the merged 3-dataset combined corpus (28,339 total samples across train/val/test) **Eval command:** `python3 -m ml.eval.evaluate --model <checkpoint> --data ml/thermal_dataset.yaml --split test` (Co-DETR used MMDetection's `tools/test.py` with an equivalent `CocoMetric` config — see Sources)

TL;DR

	YOLOv8s (production)	YOLO11x	RT-DETR-x	Co-DETR
Parameters	11.1M	56.9M	~65M	~180M (heaviest)
Training	Original 20k-image single-source set	Combined 3-dataset set, ~90 effective epochs (early-stopped, diverged after)	Combined set, 54 epochs (early-stopped, clean)	Combined set, 12 epochs (standard DETR recipe)
mAP@0.5	0.459	0.792	0.769	0.265
mAP@0.5:0.95	0.459	0.669	0.354	0.176

YOLO11x is the winner. It leads on the strict mAP@0.5:0.95 metric by a wide margin (0.669 vs RT-DETR-x's 0.354) despite RT-DETR-x scoring close on the looser mAP@0.5 (0.769 vs 0.792) — meaning RT-DETR-x finds the right region/class about as often, but its boxes are noticeably less tightly localized. Co-DETR is not competitive; see elimination note below.

Important caveat: not an apples-to-apples training comparison

`ml/checkpoints/best.pt` (YOLOv8s) was trained on the original single-source 20,000-image thermal dataset (`AxalonPIPE` -era). The three bake-off candidates were trained on the newly merged **combined** dataset (`ml/scripts/prepare_dataset.py`), which folds in two additional source datasets and substantially increases sample counts for previously rare classes (e.g. `short-circuit` went from near-absent to 124 samples).

All models were evaluated against the **same** current combined test split, which is the correct comparison for a forward-looking model-selection decision, but it means YOLOv8s is being tested on classes it essentially never saw in training — visible directly below where `string` and `short-circuit` show 0.0 AP50 for YOLOv8s specifically.

Per-class AP50

Class	Severity	YOLOv8s	YOLO11x	RT-DETR-x	Co-DETR
module	MEDIUM	0.971	0.982	0.976	0.0
cell	MEDIUM	0.799	0.778	0.759	0.0
offline-module	HIGH	0.799	0.704	0.716	0.0
vegetation-shading	LOW	0.792	0.809	0.675	0.214
cell-multi	MEDIUM	0.677	0.607	0.600	0.0
soiling	LOW	0.695	0.595	0.637	0.0
bypass-diode	CRITICAL	0.185	0.917	0.910	0.671
hot-spot-low	HIGH	0.111	0.804	0.797	0.707
string	CRITICAL	0.000	0.809	0.785	0.340
short-circuit	HIGH	0.000	0.910	0.773	0.306
hot-spot-high	CRITICAL	0.020	0.800	0.833	0.673

YOLO11x and RT-DETR-x track each other closely across almost every class (within ~0.03-0.13 AP50), consistent with both having trained on the same data and reached genuine convergence. Co-DETR scored **exactly 0.0** on five classes (`cell` , `cell-multi` , `module` , `offline-module` , `soiling`) — zero detections across every IoU threshold and object size, not just weak ones — most likely because its 12-epoch DETR-style schedule was too short to learn those categories from scratch (DETR-family models are known to converge much more slowly than YOLO-family CNNs, especially adapting COCO-pretrained weights to an unrelated 11-class thermal taxonomy).

Per-class precision / recall (YOLOv8s / YOLO11x / RT-DETR-x)

Class	YOLOv8s P/R	YOLO11x P/R	RT-DETR-x P/R
cell	0.737 / 0.768	0.665 / 0.805	0.753 / 0.695
cell-multi	0.777 / 0.404	0.592 / 0.535	0.638 / 0.543
module	0.602 / 0.974	0.930 / 0.974	0.946 / 0.939
string	1.000 / 0.000	0.806 / 0.749	0.847 / 0.669
bypass-diode	0.952 / 0.180	0.864 / 0.861	0.919 / 0.865
offline-module	0.690 / 0.723	0.564 / 0.717	0.380 / 0.795
vegetation-shading	0.733 / 0.720	0.725 / 0.713	0.578 / 0.637
soiling	0.857 / 0.599	0.920 / 0.350	0.760 / 0.600
short-circuit	1.000 / 0.000	0.684 / 0.895	0.780 / 0.684
hot-spot-low	0.727 / 0.103	0.792 / 0.737	0.813 / 0.738

Class	YOLOv8s P/R	YOLO11x P/R	RT-DETR-x P/R
hot-spot-high	0.516 / 0.024	0.741 / 0.754	0.848 / 0.776

Co-DETR precision/recall are not reported here — MMDetection's `CocoMetric` doesn't emit them in the same per-class form as Ultralytics; see `ml/eval/results/codetr_test_metrics.json` for its raw AP breakdown.

Note `soiling` : RT-DETR-x's recall (0.600) sits between YOLOv8s (0.599) and YOLO11x (0.350) — YOLO11x is uniquely conservative on this one class regardless of architecture generation, worth investigating (likely a data/label-imbalance issue rather than an architecture issue, since both newer models trained on identical data).

Why Co-DETR was eliminated from further investment

Zero detections on 5 of 11 classes is not "needs more tuning" — it means the query heads never learned to fire on those categories. Getting it competitive would need several multiples of the current 12-epoch budget, but RT-DETR-x already outperformed Co-DETR's final result by epoch ~30 of its own run, and Co-DETR's architecture (two-stage collaborative-hybrid, ~836MB checkpoints vs YOLO11x's 114MB) is far more expensive to train and to serve. Decision: no further GPU time invested in Co-DETR; its result stands as a documented data point only.

Severity-weighted read

YOLO11x and RT-DETR-x both close nearly all of YOLOv8s's gap on CRITICAL/HIGH classes (`bypass-diode` , `string` , `hot-spot-high` , `short-circuit` , `hot-spot-low`) — the fault categories that matter most operationally. Between the two, YOLO11x's much stronger mAP@0.5:0.95 means its bounding boxes are more precisely localized, which matters directly for downstream panel-level GPS localization accuracy.

Sources

- YOLOv8s eval: `ml/eval/results/20260702_123605/metrics.json`
- YOLO11x eval: `ml/eval/results/20260702_122435/metrics.json`
- RT-DETR-x eval: `ml/eval/results/20260702_223709/metrics.json`
- Co-DETR eval: `ml/eval/results/codetr_test_metrics.json` (raw log: `ml/eval/results/codetr_eval_raw.log`)
- Checkpoints: `ml/checkpoints/candidates/{yolo11x_best.pt, rtdetr_x_best.pt, codetr_epoch12.pth}`
- Design spec: `docs/superpowers/specs/2026-07-01-thermal-model-retraining-bakeoff-design.md`

Status

Bake-off complete. YOLO11x is the leading candidate for production; RT-DETR-x is a close second on mAP@0.5 but clearly behind on the stricter metric. Next step: run `ml/eval/select_winner.py` for the formal CRITICAL-weighted score, then get user confirmation before overwriting `ml/checkpoints/best.pt`.